NovaHearth Warranty Case Agent - Project Manifest

Purpose and scope

This project is a local demonstration for importing warranty claims, collecting evidence, applying policy, recording simulated actions, and routing uncertain claims to a person. The action tools do not issue real refunds or credits, reserve stock, or ship products. An in-house model endpoint is not bundled: the operator must configure an OpenAI-compatible endpoint, or select another configured provider, before claims can be processed.

Agent and orchestration logic

- **Claim intake -** `warranty_agent/agents.py`**,** `ClaimIntakeAgent`**:** Calls the selected chat model with a structured `ClaimUnderstanding` schema. It extracts the requested remedy, any product named in the message, a verbatim issue excerpt, issue category, repeat-contact indication, and whether review is needed. It does not decide warranty coverage or authorize an action. Code checks that extracted product text and excerpts are present in the customer's claim and normalizes explicit remedy contrasts.
- **Warranty assessment -** `warranty_agent/agents.py`**,** `WarrantyAssessmentAgent`**:** Uses model tool calls for read-only evidence gathering. The tool protocol requires an exact-reference warranty lookup followed by a coverage check; replacement history is required when intake identifies a replacement request. The model is prompted to follow this sequence. Code rejects malformed, repeated, and unexpected calls, and refuses to run coverage before a warranty record is loaded. It rebuilds tool arguments from the stored claim, CSV record, and policy rather than trusting model-supplied arguments. If required evidence calls are missing, code runs only those missing calls through a logged deterministic fallback. It compares the claimant with the warranty holder and assembles a concise evidence record.
- **Resolution policy -** `warranty_agent/agents.py`**,** `ResolutionPolicyAgent`**:** This is deterministic Python logic, not an LLM. It decides whether to auto-resolve or pause for review based on record availability, claimant/product match, interpretation and coverage results, eligibility, value, repeat replacement history, and policy approval flags. The action type, amount, and product come from its proposal; the action node also receives claim identifiers and, for human approvals, reviewer metadata. The LLM does not directly select or invoke the resolution action.
- **LangGraph orchestration -** `warranty_agent/workflow.py`**,** `WarrantyWorkflowFactory`**:** Routes intake -> assessment -> policy -> either action or a human approval interrupt -> action or denial. A SQLite checkpointer supports resuming a paused review. Resolution tools are not bound to the model; the graph invokes them only after the policy gate permits automation or a reviewer approves. Intake and assessment nodes allow up to two attempts for exceptions other than provider rate-limit errors; the retry predicate does not distinguish transient from permanent non-rate-limit errors.
- **Shared schemas and providers:** `warranty_agent/domain.py` defines the structured intake fields, graph state, and validated policy schema. `warranty_agent/providers.py` adapts an operator-configured OpenAI-compatible in-house endpoint, Groq, OpenAI, or local Ollama. The adapter is present, but the service URL, credentials, and model are deployment configuration, not bundled services.

Tool definitions

Read-only evidence tools

Defined with LangChain's `@tool` in `warranty_agent/agents.py`:

- `lookup_warranty_record(product_ref)` - returns the warranty row for the claim's exact product reference. The model cannot redirect the lookup using a product name from free text.
- `lookup_replacement_history(customer_name)` - combines the replacement-history CSV count with qualifying replacement actions already recorded by the application. It is required for replacement intent.
- `check_warranty_support(record_category, alleged_category, allowed_categories, issue_text)` - deterministically checks the record category against policy, the issue evidence and category, and explicit signs of accidental damage or normal wear. A known-component recall has a specific eligibility rule. The caller supplies these arguments from trusted application state.

The model sees only these evidence tools. It is constrained to the prescribed calls and a four-iteration loop; code validates calls and provides the missing-tool fallback.

Simulated action tools

Defined in `warranty_agent/action_tools.py`; invoked by the graph after its gate:

- `issue_simulated_refund(...)` - records a simulated refund.
- `issue_simulated_credit(...)` - records a simulated credit.
- `schedule_like_for_like_replacement(...)` - records a simulated replacement schedule for the authoritative product reference.

All three call `ClaimRepository.add_action`. They do not connect to payment or fulfillment systems. A human denial of an agent-completed action appends a simulated reversal row and audit/review events; it preserves the original action record and cannot undo a real-world transaction.

Data, persistence, and interfaces

- **CSV loading and validation:** `warranty_agent/data_sources.py` loads `data/warranty_claims.csv`, `data/product_warranty_records.csv`, and `data/replacement_history.csv`; checks required headers, identifiers, monetary values, and flags; and indexes warranty and replacement data.
- **Application service:** `warranty_agent/service.py` imports claims, atomically begins a run through the repository, invokes or resumes the graph, and persists the final decision state.
- **SQLite repository:** `warranty_agent/repositories.py` creates and migrates the tables `claims`, `actions`, `human_reviews`, `events`, and `llm_usage`. It records decisions, proposals, simulated actions, reviewer decisions, token usage, and audit events; it also handles action idempotency, stale-run recovery, clear-and-reseed, and simulated reversals. LangGraph's SQLite checkpointer stores workflow checkpoints in the configured database.
- **Token accounting:** `warranty_agent/token_usage.py` reads usage metadata returned by providers and stores it; missing provider usage is left unreported rather than estimated.
- **HTTP API and dashboard:** `api.py` exposes health/configuration, claims, import, processing/batches, reviews, overrides/reversals, actions, audit events, usage, warranties, and demo reset routes. `frontend/src/App.jsx`, `frontend/src/api.js`, and `frontend/src/styles.css` implement the React dashboard, including claim detail, agent-completed, human-review, action, audit, usage, and policy views.
- **Configuration and verification:** `policy.json` stores the policy version, supported categories, approval flags, and the default auto-resolution limit. `tests/test_warranty_app.py` covers policy boundaries, tool calls/fallback, prompt-injection containment, concurrency, review, reset, provider configuration, and persistence.

Design Rationale

I kept the workflow explicit because each claim needs an auditable path and uncertain claims must pause for human review. The LLM helps interpret free text and request evidence, but it cannot choose the warranty record, approve a claim, set the value, or execute a resolution. I used the claim's product reference for exact lookup because an incorrect product match could lead to an incorrect action. The deterministic policy gate and SQLite ledger make decisions and actions inspectable. This is a demo, so CSV and SQLite are straightforward; they are not choices I would keep unchanged for production scale.

The brief does not define "reasonable value" or how to value a replacement. I made the cap configurable and chose a demo default of **strictly below $100**. Refunds and credits use the record's credit value; replacements use purchase price as the value exposure. Values at or above the limit, unclear evidence, mismatches, and repeat replacement reports go to a human. I show evidence and concise reasons rather than private model chain-of-thought. The in-house provider is an adapter only: the operator still needs to supply a compatible endpoint and credentials if required. Actions are simulated, so the demo does not claim that money moved or a unit shipped.

During an earlier AI-assisted iteration, a model self-reported confidence field was displayed as if it might help assess a claim. I questioned its value: the brief does not ask for confidence, and a model's own score is not calibrated evidence. I removed it and kept approval tied to verifiable claim text, tool results, and deterministic policy. With more time, I would test a held-out labeled set, refine retries to target transient errors, add authentication and reviewer roles, use a durable worker queue, and integrate idempotent payment/fulfillment services with reconciliation and monitoring.